Meeting #3

2026, March 3nd

Setup

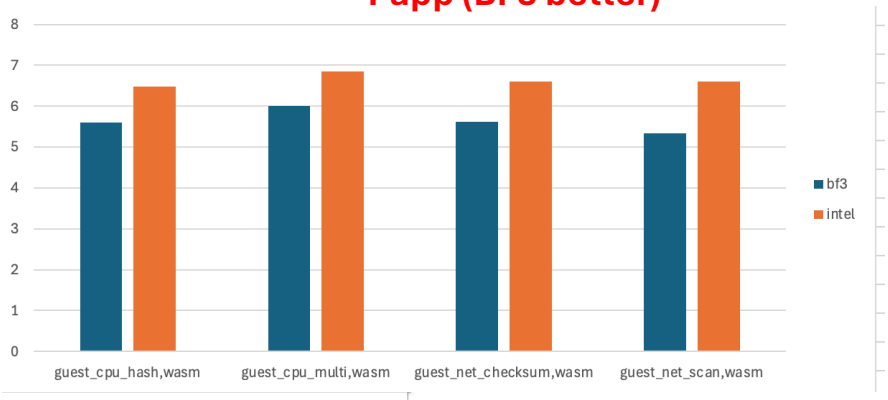
- The host application is implemented in Rust and runs Wasmtime with WASI support, exposing an HTTP endpoint per instance (distinct ports) -- (used Tokio lib and Hyper).
- Each request triggers a fresh WebAssembly instantiation and invocation of the guest handle function.
- Guest applications are compiled to wasm32-wasip1.
- Workload is generated using wrk:
    - Number of concurrent host instances (1–10 ports)
    - Number of flows (50–1000 connections)

Guest aplications

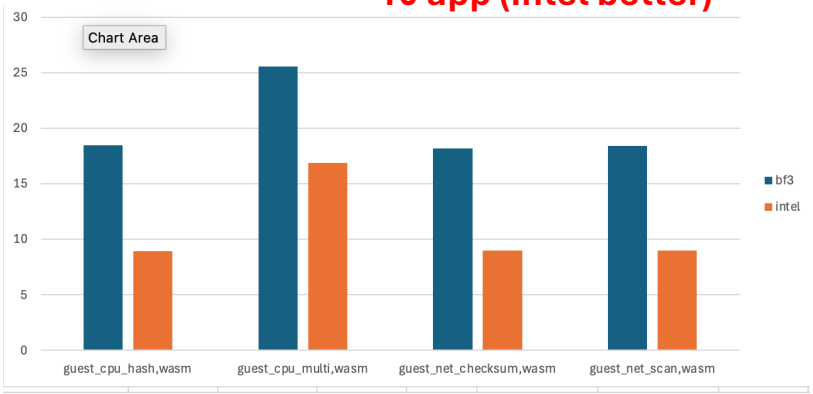
Guest Application	Workload Type	Primary Stress Target	Microarchitectural Signature	Expected perf Behavior
`guest_cpu_hash.wasm`	CPU-bound hashing	Integer ALUs, pipelines	High instructions & cycles, moderate IPC, moderate branches	High CPU usage, low cache miss rate
`guest_cpu_multi.wasm`	Arithmetic loops	Execution width, throughput	Very high instructions, stable IPC, minimal memory pressure	CPU saturation, strong scaling until core contention
`guest_mem_scan.wasm`	Sequential memory	Cache hierarchy (L1/L2/L3)	Increased cache references, moderate miss rate	Lower IPC if working set exceeds cache
`guest_mem_random.wasm`	Random access	Memory latency, DRAM stalls	Higher cache miss rate, reduced IPC	Memory-bound behavior
`guest_net_checksum.wasm`	Streaming checksum	Memory bandwidth + ALU mix	Moderate instructions, low branch miss	Balanced CPU/memory load
`guest_net_scan.wasm`	Pattern scanning	Branch predictor, control flow	Elevated branches and branch-miss rate	Lower IPC if misprediction increases

avarage latency: bf3 vs intel-xeon (wrk)

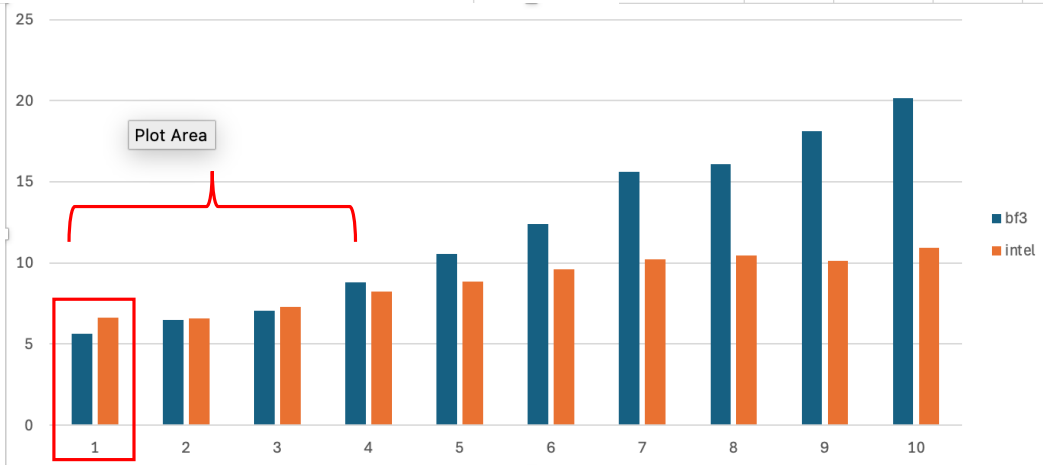
1 app (BF3 better)



10 app (Intel better)

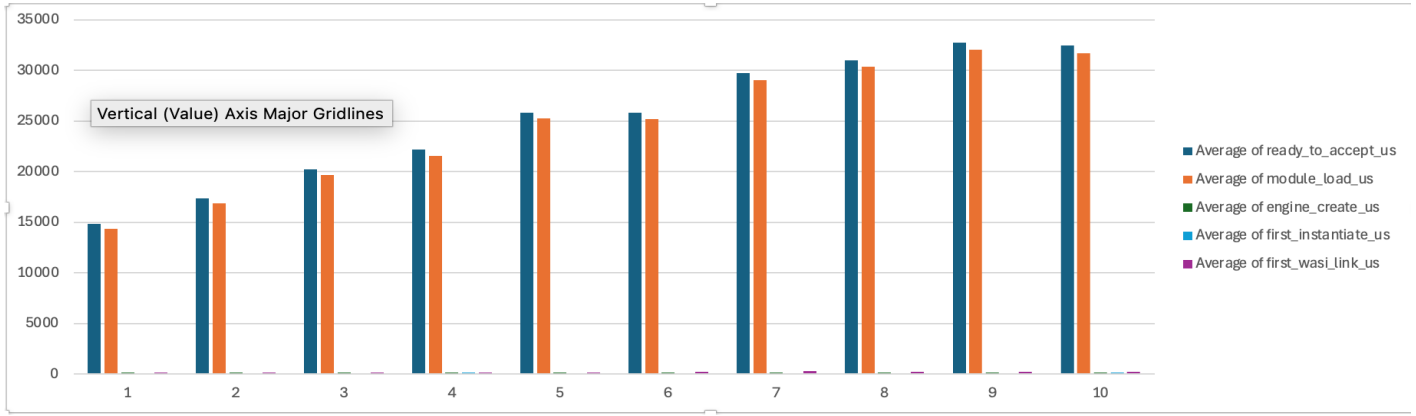


Envoy: - BF3 22ms
- Intel 6ms

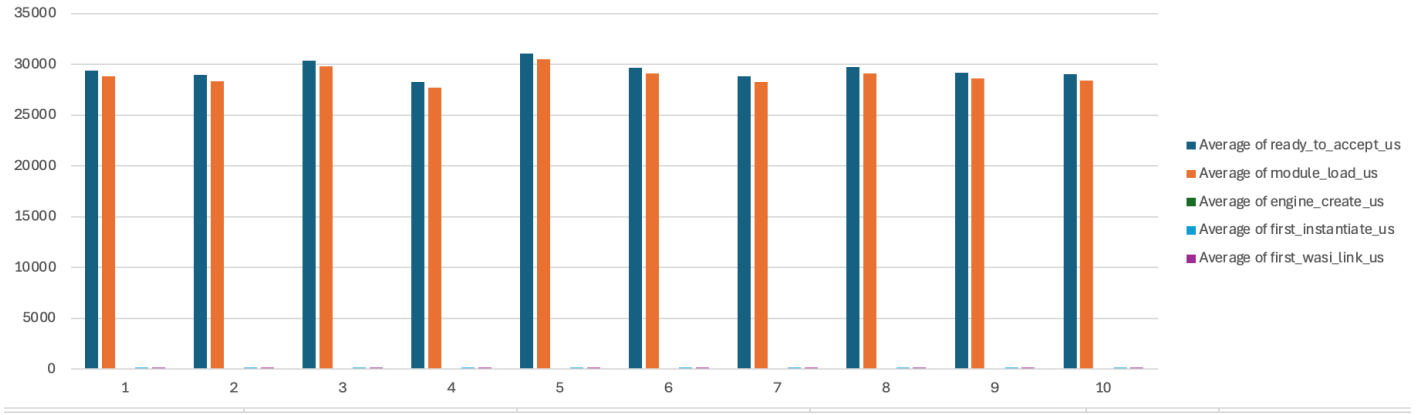


cold-start latency: bf3 vs intel-xeon

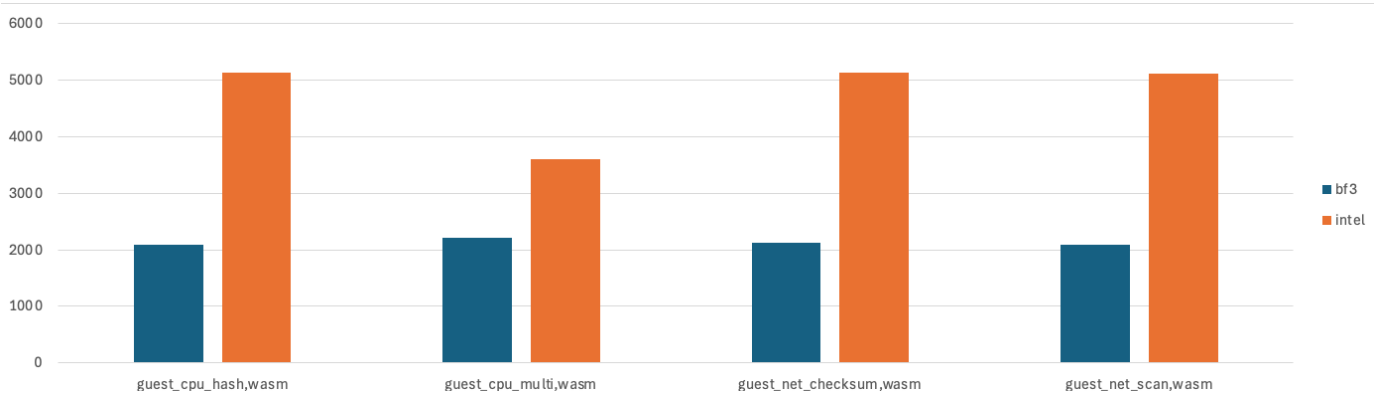
Intel



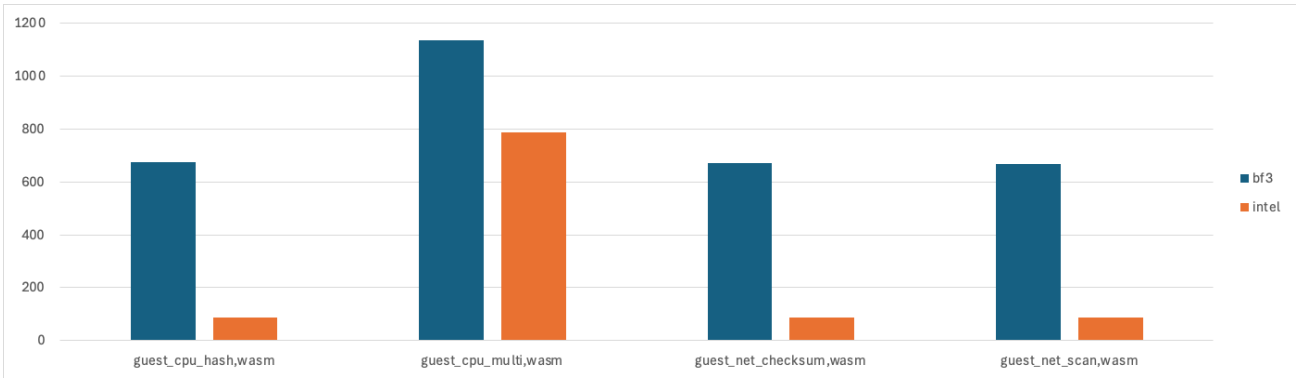
BF3



per-request metrics: bf3 vs intel-xeon

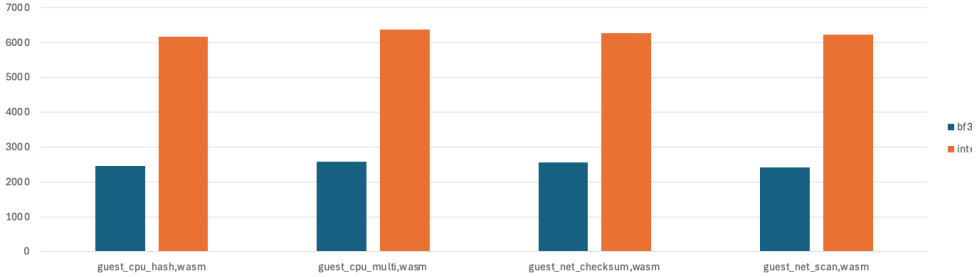


Instantiate

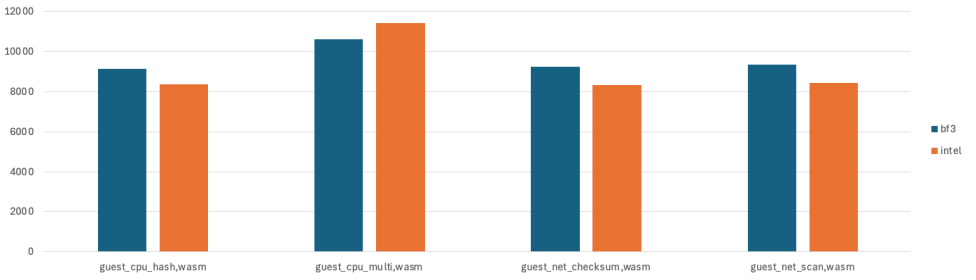


handle request

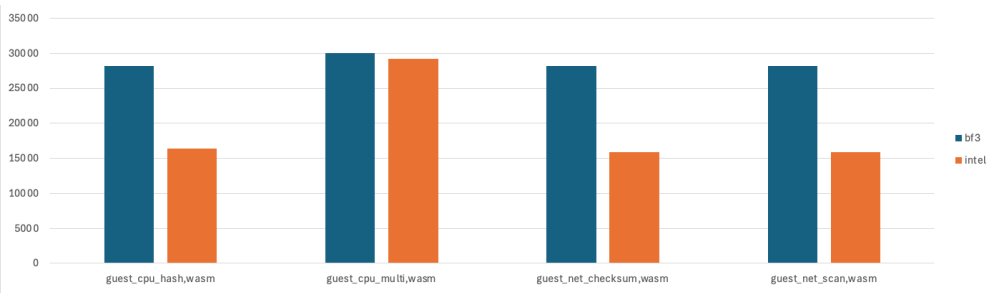
per-request metrics: bf3 vs intel-xeon



Total (1 app)



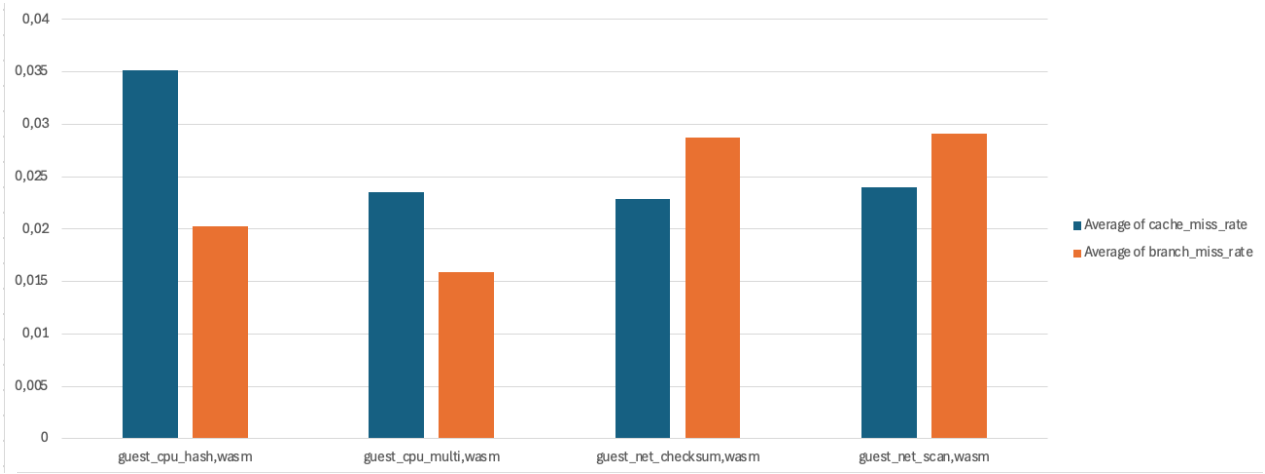
Total (5 app)



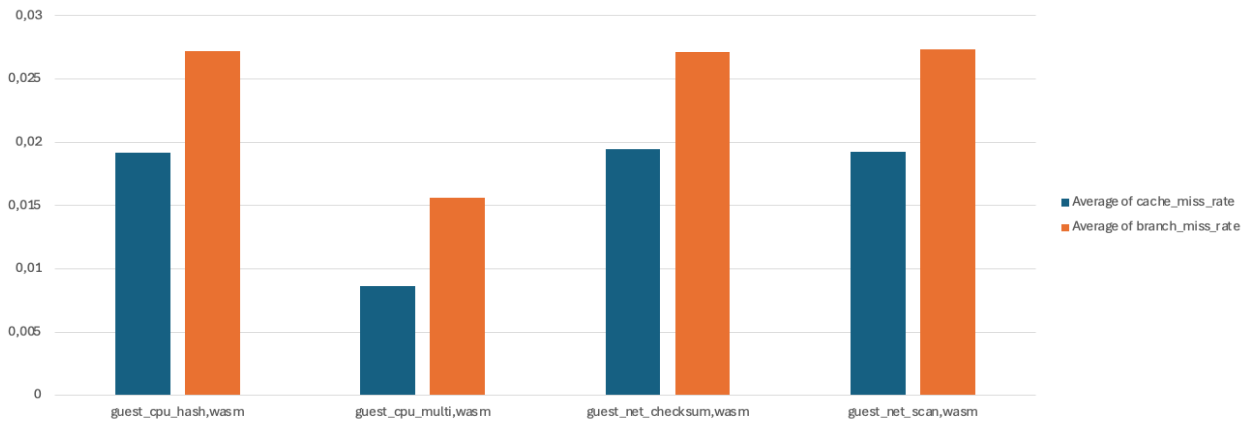
Total (10 app)

cache/branch misses: bf3 vs intel-xeon

Intel

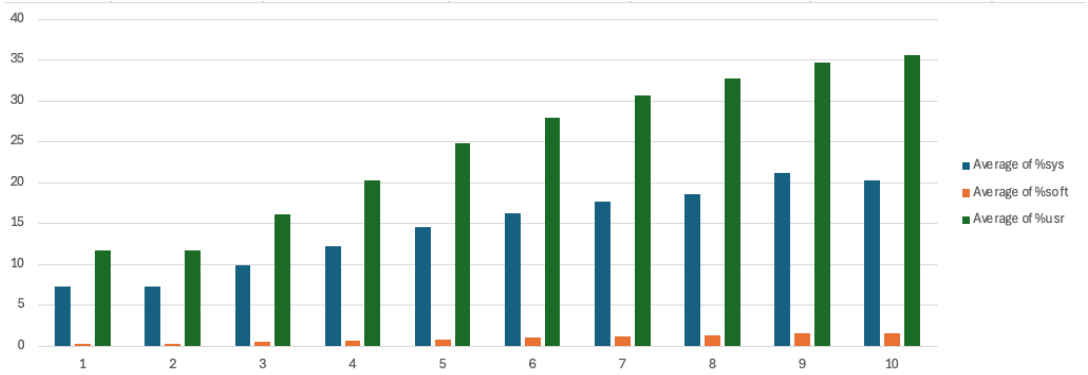
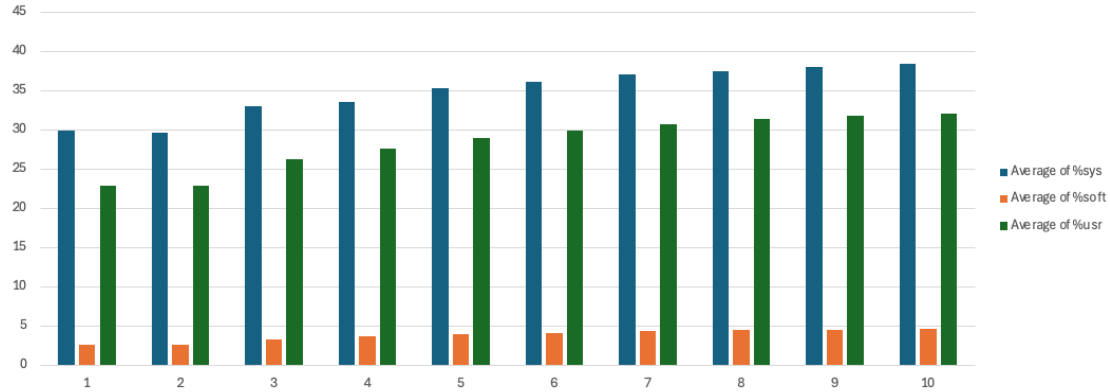


BF3



cpu-metrics: sar

Intel



BF3